A3-PS001-TC454

Android Ransomware-like Activity Early Warning in a Controlled Lab

Soutri Bose
soutribose321@gmail.com

Kaushiki Chowdhury
dgp.kau@gmail.com

Institution
Institute of Engineering & Management, Kolkata

Mentor
Dr. Prithwineel Paul
prithwineelpaul@gmail.com

AIORI Hackathon 3
October 13, 2026

Problem Description

Ransomware encrypts files faster than humans can react — detection has to be early, lightweight and explainable.

- **Damage happens in seconds.** Crypto-ransomware reads, encrypts, then deletes or overwrites user files; after encryption the data is already lost, so the goal is early warning, not after-the-fact classification.
- **ML-heavy detectors are costly on phones.** Most Android proposals classify apps from permissions, API calls or traffic using ML, with high CPU, memory and battery cost, often processed off-device [1][7].
- **Single-signal detection is brittle.** Honeyfile-only monitoring can miss ransomware that avoids decoys, and hard-coded access patterns risk false alerts on benign bulk operations [1].
- **Platform limits.** Android 11+ scoped storage restricts what an app can observe, and lock-screen ransomware needs a separate detection branch [11].

PS-001 objective

Build and demonstrate, in a controlled lab, a system that raises an early, explainable warning on ransomware-like file activity.

Success targets (lab)

- Warning within ≤ 2 s of the attack starting
- ≤ 10 of 100 test files modified before the warning
- 0 false alarms across 20 bulk operations (4 workloads × 5 runs)
- Cost on the phone: CPU ≤ 3 %, memory ≤ 50 MB, battery ≤ 1 %/h, ≤ 50 ms per event

Background

What is needed to understand the problem and the proposed optimization.

- **How ransomware touches files [1].** Three recurring patterns: overwrite, read–encrypt–delete, read–encrypt–overwrite. Any sample must access and manipulate files, legitimate or decoy.
- **Reactive monitoring.** Android's FileObserver wraps Linux inotify: the kernel notifies the app on OPEN, ACCESS, MODIFY, CREATE, DELETE, MOVED_*, so there is no polling and idle cost is near zero [1][9].
- **Canary (honey) files.** Decoy files nobody legitimate should touch; tampering is a high-fidelity alert.
- **Shannon entropy [10].** Encrypted data approaches 8 bits/byte, but compressed files do too, so entropy is supporting evidence only.
- **Magic-byte check.** If a file's header stops matching its type after a write, that is a cheap, strong encryption signal and makes entropy far more trustworthy.

Signals the system fuses

FileObserver
Rapid write / delete / rename bursts

Canary files
Decoy tampering, SHA-256 mismatch

Entropy
Encryption-like content change

Platform limits we design around

- **Scoped storage:** only a dedicated directory is monitored [11]
- **No recursion:** one FileObserver per subfolder
- **No app attribution:** we can warn, not kill the process
- **Queue overflow:** inotify can drop events in a burst; rescan and alert

Literature & RFC Review

Most relevant proposals and platform specifications only.

Work	Technique	Takeaway for this project
R-Locker [2]	FIFO honeyfile blocks a reader process (Linux/Windows)	Honeyfile idea is sound, but FIFOs and links are not feasible on Android external storage
Gómez-Hernández & García-Teodoro 2024 [1]	Honeyfiles + reactive FileObserver on Android	Principal reference: 1–200 ms notification for 200 events (1 app); ~10 files lost with 60 KB decoys. Patterns hard-coded; few samples
Moussaileb et al. [3]	Decoy folders; alert when a normalized access count exceeds a threshold	Burst/threshold counting works, but a fixed threshold is brittle
RWGuard [4], RansomWall [5]	Decoys combined with I/O monitoring in a multi-layer defence	Decoys + I/O monitoring are already combined, so fusion alone is not our novelty. Ours: unprivileged, on-device, explainable, no ML
KRProtector [8]	Decoy-based file protection on Android without root	Closest unprivileged Android decoy design. We add an explainable score and measured cost
RansomCare [6]	Data-entropy-based detection and mitigation (~1.4 s)	Entropy is a useful content signal
RanDetector [7]	ML on permissions, intents, APIs (>97.6%)	Accurate but ~45 s and resource-heavy; not an early warning
FileObserver API, scoped storage [9][11]	Platform specification (no IETF RFC governs this)	Defines observable events and the dedicated-directory scope

Proposed Solution

Detection logic: three signals plus a file-type check, fused into one transparent risk score.

Risk score per 5 s window (every term in [0, 1])

$$R = 0.35 \cdot burst + 0.35 \cdot canary + 0.15 \cdot \Delta H + 0.15 \cdot magic$$

- **burst** = min(z, 6) / 6, with z = (n − μ) / σ for the window's event count n
- **Adaptive threshold:** μ and σ are learned over a 5-minute learning phase (60 normal windows); burst alert when n > μ + 3σ
- **canary** = 1 if any decoy's SHA-256 changes or it is renamed or deleted
- **ΔH** = min((H_after − H_before) / 4, 1), entropy of changed blocks in bits/byte
- **magic** = 1 if a file's header no longer matches its type after a write

Worked example

Scenario	burst	canary	ΔH	magic	R	Level
Attack: n = 32, μ = 8, σ = 4, H 4.1 → 7.9	1.00	1	0.95	0	0.84	HIGH
Bulk copy of 500 photos	1.00	0	0	0	0.35	Log only

Weights and thresholds are initial values, calibrated on lab traces and frozen before the evaluation.

Risk levels ($\theta_1 = 0.65$, $\theta_2 = 0.40$)

HIGH	$R \geq \theta_1$: e.g. canary hit + burst. Notify at once
MEDIUM	$\theta_2 \leq R < \theta_1$: burst + content change, canaries intact
LOW	$R < \theta_2$: log only. No single signal can reach θ_2

Design rule: behavior is the primary detector, canaries are strong, entropy is supporting. Never alert on entropy alone.
After an alert: a notification with risk level, evidence, timestamp and affected files, plus an optional snapshot copy of files not yet touched.

Threat Model & Evasion Handling

Four ways to get around or fool the detector, and how each is answered up front.

Case	What goes wrong	Countermeasure
Slow encryption	Stays under the per-window burst threshold	Cumulative counter of changed files over a longer window (10 min), with its own learned threshold
Partial encryption	Only the first KB of each file changes, so whole-file entropy barely moves	Compare entropy before and after on the changed blocks, not the whole file
Skipping the decoys	Ransomware avoids known honeyfiles	Canaries in every subfolder, realistic sizes, named to sort first (e.g. !000_…). Burst and entropy still score without them
High-entropy benign files (zip, jpg, mp4)	Entropy alone raises false alarms	Magic-byte check: a header that stops matching its type after a write adds weight. Entropy alone can never reach θ_2

Attacker model. An unprivileged app with storage access that may know we monitor, but cannot modify our app's data or stop its service.
Out of scope. Root-level or kernel ransomware and lock-screen ransomware (later extension).

Optimization Proposed

Versus [1], RWGuard [4], RansomWall [5] and KRProtector [8]: ours is unprivileged, on-device, explainable and ML-free.

Aspect	Before (baseline)	After (proposed)	Expected benefit
Detection signal	Honeyfile access via FileObserver only	Risk score fusing canary integrity, operation bursts and entropy	Fewer misses when decoys are avoided
Detection rules	Access patterns embedded in app code	Transparent weighted formula (slide 5), weights and thresholds in config	Tunable and auditable
Thresholds	Fixed	Adaptive: mean + 3σ of events per window, learned in a 5-minute phase on normal activity	Bulk copy / move does not raise warnings
Content check	None	Block-level entropy + magic-byte check + SHA-256 canary hashes	Separates encryption-like change from edits; never alert on entropy alone
Alert output	Raw event list per path	Explainable alert: risk level, evidence, timestamp, affected files	Faster triage
Evaluation	Few samples; notification time reported	Benign simulator: latency, files lost, false alarms, plus CPU, memory, battery	Measurable early-warning claim

Expected benefits are design goals to be verified in the lab evaluation, not measured results.

Solution Architecture & Design

Visual: component view of the detection pipeline. Labelled data flows are on the next slide (DFD).

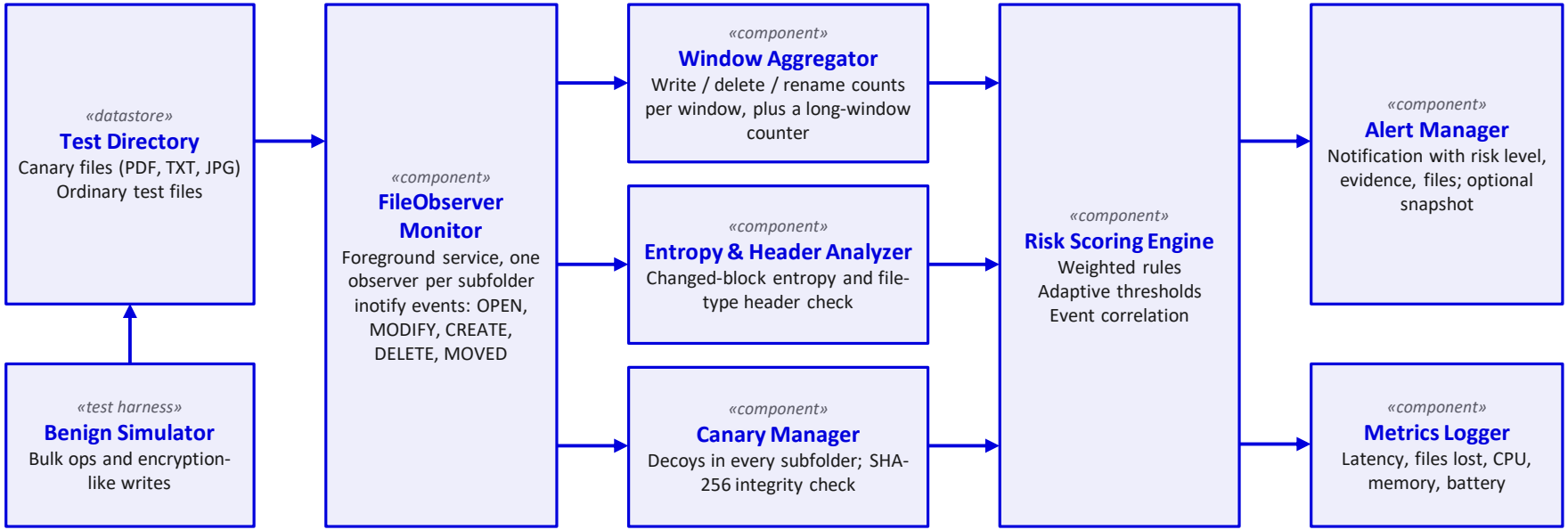


Figure: System Architecture UML Diagram (component view)

Data Flow Diagram

Overview of the architecture: how file events become evidence, a risk score and an alert.

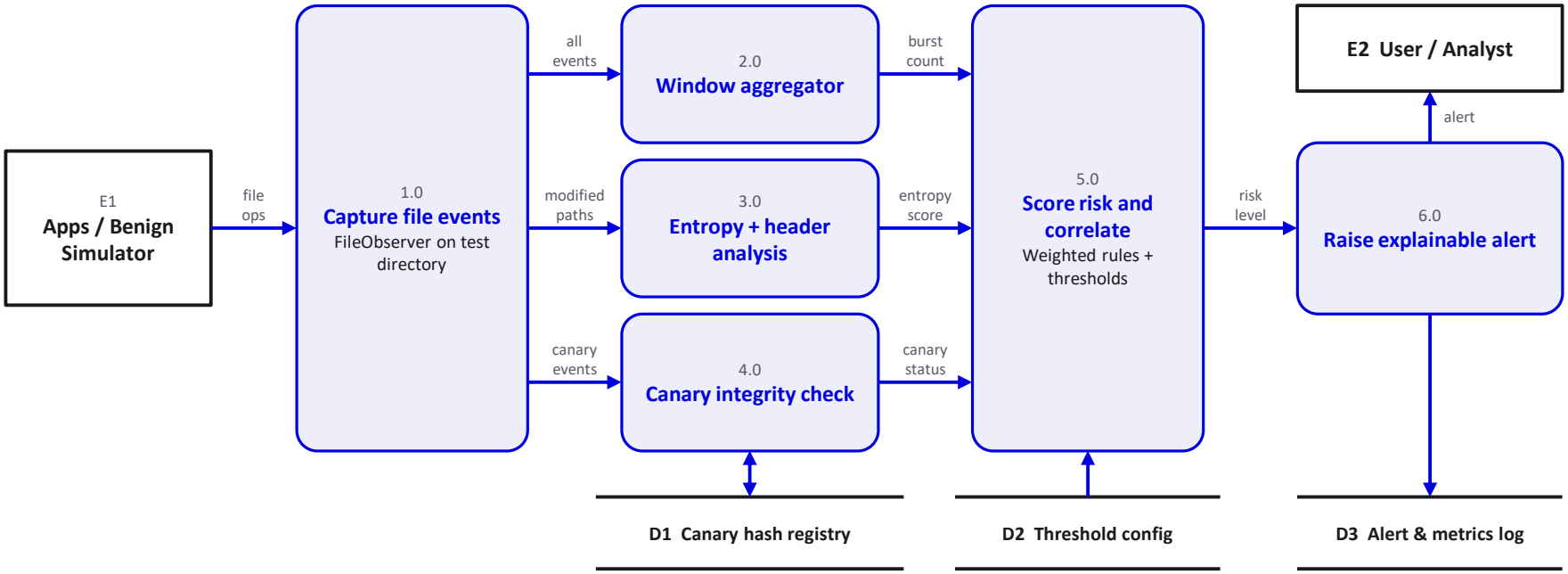


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Five sequential phases; each ends with a demonstrable milestone.



Monitoring

Kotlin app + foreground service; one observer per subfolder

Milestone: Events logged live

Canary traps

Canaries in every subfolder; SHA-256 baselines

Milestone: Tamper detected

Detection

Window and cumulative counters, entropy, magic-byte check

Milestone: Signals extracted

Risk scoring

Implement the risk formula; learn μ, σ; calibrate weights

Milestone: Explainable alert

Evaluation

Attack and benign scenarios; latency, files lost, false alarms, cost

Milestone: Results table

Scope. Crypto-ransomware on an Android 11+ emulator and a rooted test device. Rule-based first; Random Forest only if false positives demand it.
Demo claim. Attack flagged within 2 s with ≤ 10 of 100 files modified, and 0 false alarms on 20 bulk operations (targets).
Later extensions. Lock-screen / device-admin detection branch; root-assisted system-call tracing.

Evaluation Plan

Planned lab study. No results are claimed yet: every figure below is a target.

Attack scenarios

- The three patterns from [1]: overwrite, read–encrypt–delete, read–encrypt–overwrite
- Partial encryption (first KB only)
- Slow encryption (below the burst threshold)

Benign workloads

- Bulk copy / move
- Zip / unzip
- Photo backup
- Media transcoding

Metric	Target	Status
Detection latency	≤ 2 s from attack start	Planned
Files modified before warning	≤ 10 of 100 test files	Planned
False alarms	0 across 20 bulk operations	Planned
CPU while monitoring	≤ 3 % average	Planned
Memory	≤ 50 MB	Planned
Battery drain	≤ 1 % per hour extra	Planned
Event-handling latency	≤ 50 ms per event (p95)	Planned
Overflow recovery	Alert after a forced inotify overflow	Planned

Each scenario: 5 runs on 100 disposable files. Reference point from [1]: about 10 files lost with 60 KB decoys, 1–200 ms notification time.

Bibliography

[1] J. A. Gómez-Hernández and P. García-Teodoro. “Lightweight Crypto-Ransomware Detection in Android Based on Reactive Honeyfile Monitoring.” In: Sensors 24.9 (2024), 2679. doi:10.3390/s24092679
[2] J. A. Gómez-Hernández, L. Álvarez-González, P. García-Teodoro. “R-Locker: Thwarting Ransomware Action through a Honeyfile-based Approach.” In: Computers & Security 73 (2018), pp. 389–398.
[3] B. Moussaileb et al. “Ransomware’s Early Mitigation Mechanisms.” In: Proc. ARES (2018), pp. 1–10.
[4] S. Mehnaz, A. Mudgerikar, E. Bertino. “RWGuard: A Real-Time Detection System Against Cryptographic Ransomware.” In: RAID 2018, LNCS 11050.
[5] S. K. Shaukat and V. J. Ribeiro. “RansomWall: A layered defense system against cryptographic ransomware attacks using machine learning.” In: Proc. COMSNETS (2018), pp. 356–363.
[6] F. Faghihi and M. Zulkernine. “RansomCare: Data-centric detection and mitigation against smartphone crypto-ransomware.” In: Computer Networks 191 (2021), 108011.
[7] A. Alzahrani et al. “An Intelligent Behavior-Based Ransomware Detection System For Android Platform.” In: Proc. IEEE TPS-ISA (2019), pp. 28–35.
[8] S. Wang et al. “KRProtector: Detection and Files Protection for IoT devices on Android without ROOT against Ransomware Based on Decoys.” In: IEEE Internet of Things Journal 9 (2022), pp. 18251–18266.
[9] Android Developers. “FileObserver abstract class.” developer.android.com/reference/android/os/FileObserver (accessed 14 Oct 2026)
[10] C. E. Shannon. “A Mathematical Theory of Communication.” In: Bell System Technical Journal 27 (1948), pp. 379–423.
[11] Android Developers. “Storage updates in Android 11.” developer.android.com/about/versions/11/privacy/storage (accessed 14 Oct 2026)

Thank You

Demo / code: [add repository link]

Soutri Bose: soutribose321@gmail.com
Kaushiki Chowdhury: dgp.kau@gmail.com

Faculty Mentor : Dr. Prithwineel Paul : prithwineelpaul@gmail.com

A3-PS001-TC454 · AIORI Hackathon 3